Spring Boot Guide

REST APIs, application layers, configuration, validation, and testing

Purpose: This document is a sample knowledge source for the DevPilot AI RAG ingestion pipeline. It is written with headings and concise sections so the service can extract text, create chunks, embed them, and retrieve relevant passages.

1. What Spring Boot Provides

Spring Boot simplifies the creation of stand-alone, production-oriented Spring applications. It provides auto-configuration, starter dependencies, externalized configuration, and convenient application startup.

Auto-configuration

Auto-configuration configures common components based on the libraries and properties present on the classpath. Developers can override defaults when their application requires different behavior.

Starter dependencies

Starters group related dependencies. For example, a web starter provides the components needed to build HTTP APIs, while a data JPA starter provides persistence integration.

2. Typical REST API Layers

Layer	Responsibility	Example
Controller	HTTP mapping and request/response handling	ExpenseController
Service	Business rules and transaction boundary	ExpenseService
Repository	Database access	ExpenseRepository
Entity	Persistent data model	Expense
DTO	External request/response shape	CreateExpenseRequest

Keep business rules in services rather than placing all logic in controllers. DTOs help avoid exposing persistence entities directly as public API contracts.

3. A Simple Controller Example

```
@RestController\n@RequestMapping("/api/expenses")\nclass ExpenseController {\n    private final Ex
penseService service;\n\n    ExpenseController(ExpenseService service) {\n        this.service = s
ervice;\n    }\n\n    @GetMapping\n    List<ExpenseResponse> findAll() {\n        return service.f
indAll();\n    }\n}
```

The example illustrates constructor injection and request mapping. In a real application, define the DTO and service methods, apply authorization, and return appropriate HTTP status codes.

4. Dependency Injection and Configuration

Spring manages application objects as beans. Constructor injection makes dependencies explicit and improves testability. Configuration should be externalized into application.yml and environment variables.

```
spring:\n  datasource:\n    url: ${DB_URL}\n    username: ${DB_USERNAME}\n    password: ${DB_PASSW
ORD}
```

Do not commit database passwords, JWT signing secrets, or provider API keys. Environment variables are a convenient local development option; production deployments should use an approved secret-management mechanism.

5. Validation and Error Handling

Use Jakarta Bean Validation annotations such as @NotBlank, @Email, @Size, and @Positive on request DTOs. Add @Valid to controller parameters to trigger validation.

Use a centralized exception handler, commonly with @RestControllerAdvice, to return consistent client-safe errors. Do not expose stack traces, SQL details, secrets, or internal class names in API responses.

```
public record CreateExpenseRequest(\n    @NotBlank String description,\n    @Positive BigDecimal a
mount\n) {}
```

6. Testing and Observability

Use unit tests for business logic, slice tests for MVC or persistence behavior, and integration tests for important end-to-end paths. Test both successful and failing requests.

- Verify invalid input returns a clear 400 response.
- Verify missing authentication returns 401 for protected routes.
- Verify users cannot access resources owned by another user.
- Use health and metrics endpoints carefully; expose only what is appropriate for the environment.

7. RAG Questions

- What is the responsibility of a Spring service layer?
- Why use DTOs instead of returning JPA entities directly?
- How does constructor injection improve testability?
- Where should secrets be stored?
- How should validation errors be returned to API clients?

End of document. Use this file for development and testing; verify all examples against your actual application configuration before treating them as production standards.